



MySQL

Comisión 81860

Julian Daud

PROYECTO: Ecommerce de Instrumentos Musicales

1) Descripción de la temática

El proyecto desarrolla la base de datos de un e-commerce de instrumentos musicales. El sistema permite registrar clientes, administrar un catálogo de productos, generar órdenes de compra y registrar pagos. La base de datos organiza y centraliza toda la información necesaria para el funcionamiento del sitio.

2) Situación problemática

La tienda usa archivos sueltos para manejar datos, lo que provoca:

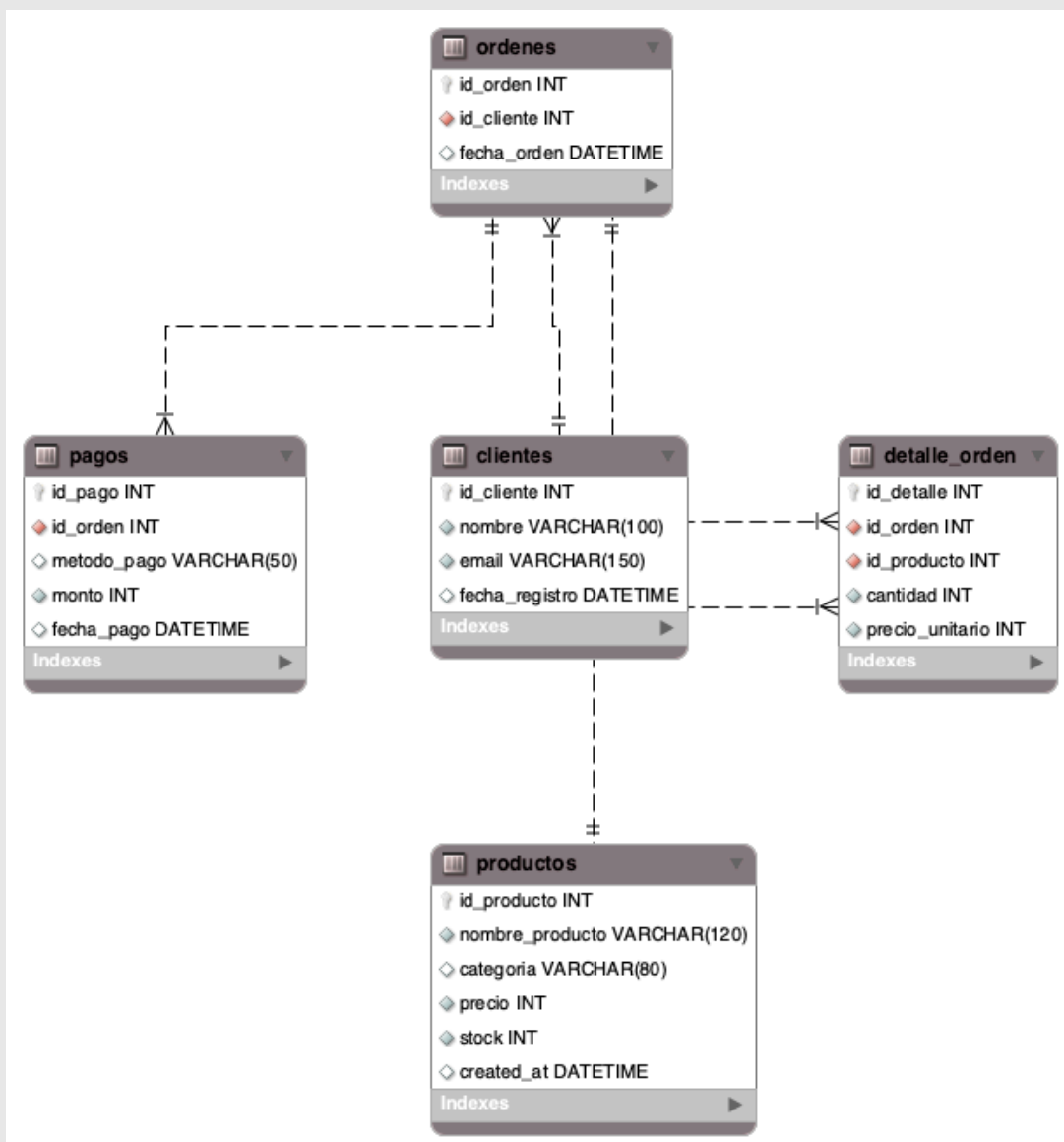
1. Dificultad para rastrear ventas.
2. Errores e inconsistencias en el stock.
3. Datos duplicados o incompletos.
4. Falta de un registro claro de pagos.

Se necesita una base de datos relacional que unifique y ordene toda la información.

3) Modelo de negocio

La empresa vende instrumentos musicales a través de un e-commerce. Ofrece productos con stock, clientes registrados, órdenes con múltiples ítems y un sistema de pagos. El negocio genera ingresos por venta directa y requiere una estructura de datos clara para operar de manera eficiente.

DIAGRAMA MODELO ER



LISTADO DE TABLAS

1) CLIENTES:

Registra los datos básicos de los clientes que se registran y compran en el e-commerce.

Nombre del campo	Abreviatura	Tipo de dato	Clave	Descripción
id_cliente	id_cliente	INT	PK	Identificador único del cliente
nombre	nombre	VARCHAR(100)	—	Nombre del cliente
email	email	VARCHAR(150)	—	Correo electrónico del cliente
fecha_registro	fecha_registro	DATETIME	—	Fecha en que el cliente se registró

2) PRODUCTOS:

Contiene el catálogo de instrumentos y accesorios disponibles para la venta

Nombre del campo	Abreviatura	Tipo de dato	Clave	Descripción
id_producto	id_producto	INT	PK	Identificador único del
nombre_producto	nombre_producto	VARCHAR(120)	—	Nombre del producto o
categoria	categoria	VARCHAR(80)	—	Tipo/categoría del instrumento
precio	precio	INT	—	Precio del producto
stock	stock	INT	—	Cantidad disponible en
created_at	created_at	DATETIME	—	Fecha en que se agregó el

3) ORDENES:

Registra cada compra realizada por los clientes

Nombre del campo	Abreviatura	Tipo de dato	Clave	Descripción
id_orden	id_orden	INT	PK	Identificador único de la orden
id_cliente	id_cliente	INT	FK	Cliente que realizó la compra
fecha_orden	fecha_orden	DATETIME	—	Fecha en que se generó la orden

4)DETALLE_ORDEN

Registra los productos incluidos en cada orden

Nombre del campo	Abreviatura	Tipo de dato	Clave	Descripción
id_detalle	id_detalle	INT	PK	Identificador único del detalle
id_orden	id_orden	INT	FK	Orden asociada
id_producto	id_producto	INT	FK	Producto incluido
cantidad	cantidad	INT	—	Cantidad comprada
precio_unitario	precio_unitario	INT	—	Precio del producto en el momento de la compra

5)PAGOS

Registra los pagos realizados por cada orden.

Nombre del campo	Abreviatura	Tipo de dato	Clave	Descripción
id_pago	id_pago	INT	PK	Identificador único del pago
id_orden	id_orden	INT	FK	Orden asociada al pago
metodo_pago	metodo_pago	VARCHAR(50)	—	Método de pago utilizado
monto	monto	INT	—	Importe del pago
fecha_pago	fecha_pago	DATETIME	—	Fecha en que se realizó el pago

Comisión 81860
Julian Daud

CORRECCIONES 1RA ENTREGA

Se realizaron ajustes en la base de datos :

```
== CORRECCIONES ==#

72 • ALTER TABLE clientes
73   ADD UNIQUE (email),
74   ADD COLUMN ciudad VARCHAR (100),
75   ADD COLUMN pais VARCHAR (100);
76
77
78 • ALTER TABLE ordenes
79   ADD COLUMN total DECIMAL(10,2);
80
81
82 • ALTER TABLE productos
83   MODIFY precio DECIMAL (10, 2);
84
85 • ALTER TABLE detalle_orden
86   MODIFY precio_unitario DECIMAL (10, 2);
```

- Clientes: **email único** y nuevas columnas **ciudad y pais**.
- Órdenes: nueva columna **total** para el monto de la orden.
- **Productos / Detalle de Orden**: precios cambiados a **DECIMAL(10,2)** para mayor precisión.

Estos cambios mejoran la integridad de los datos y permiten las nuevas funcionalidades del proyecto.

CARGA DE DATOS A LA BASE

Se muestran los comandos **INSERT** utilizados para agregar datos a las tablas, junto con screenshots de la ejecución en MySQL.

1) PAGOS

```
262 # === PAGOS === #
263
264
265 • INSERT INTO pagos (id_orden, metodo_pago, monto, fecha_pago) VALUES
266 (1, 'Tarjeta de crédito', 68.00, '2025-01-15 11:00:00'),
267 (2, 'Venmo', 79.00, '2025-02-03 15:00:00'),
268 (3, 'Zelle', 72.00, '2025-03-10 10:00:00'),
269 (4, 'Tarjeta de crédito', 87.50, '2025-04-22 17:00:00'),
270 (5, 'Venmo', 71.00, '2025-05-05 12:30:00'),
271 (6, 'Zelle', 57.50, '2025-06-18 14:00:00'),
272 (7, 'Tarjeta de crédito', 82.50, '2025-07-07 10:30:00'),
273 (8, 'Venmo', 70.00, '2025-08-12 13:30:00'),
274 (9, 'Zelle', 73.00, '2025-09-25 16:00:00'),
275 (10, 'Tarjeta de crédito', 54.00, '2025-10-03 11:00:00');
276
277
278 #=== VISTAS ===#
279
280 #1=#
281
282 • CREATE VIEW vw_detalle_compra AS
283 SELECT p.categoria,
```

4) CLIENTES

```
INSERT INTO clientes (nombre, email, fecha_registro, ciudad, pais)
VALUES
('Raul Salvatierra', 'raul.salvatierra.35@gmail.com', NOW(), 'Brooklyn', 'USA'),
('Maria Perez', 'maria@hotmail.com', NOW(), 'Buenos Aires', 'Argentina'),
('Carlos Lopez', 'carlos@gmail.com', NOW(), 'Madrid', 'España'),
('Gian Tenorio', 'gian.tenorio@gmail.com', NOW(), 'Cali', 'Colombia'),
('Brian Scheller', 'bscheller@gmail.com', NOW(), 'Brooklyn', 'USA'),
('Juan Perez', 'juan.perez@gmail.com', NOW(), 'Buenos Aires', 'Argentina'),
('Maria Gonzalez', 'maria.gonzalez@yahoo.com', NOW(), 'Santiago', 'Chile'),
('Carlos Ramirez', 'carlos.ramirez@hotmail.com', NOW(), 'Lima', 'Peru'),
('Ana Fernandez', 'ana.fernandez@gmail.com', NOW(), 'Bogotá', 'Colombia'),
('Luis Martinez', 'luis.martinez@yahoo.com', NOW(), 'Quito', 'Ecuador'),
('Sofia Lopez', 'sofia.lopez@hotmail.com', NOW(), 'Montevideo', 'Uruguay'),
('Diego Sanchez', 'diego.sanchez@gmail.com', NOW(), 'Asunción', 'Paraguay'),
('Camila Torres', 'camila.torres@yahoo.com', NOW(), 'La Paz', 'Bolivia'),
('Jose Castillo', 'jose.castillo@hotmail.com', NOW(), 'Caracas', 'Venezuela'),
('Valentina Herrera', 'valentina.herrera@gmail.com', NOW(), 'Ciudad de Mexico', 'Mexico'),
('Ricardo Morales', 'ricardo.morales@yahoo.com', NOW(), 'Guadalajara', 'Mexico'),
('Fernanda Diaz', 'fernanda.diaz@hotmail.com', NOW(), 'Medellín', 'Colombia'),
('Gabriel Ortiz', 'gabriel.ortiz@gmail.com', NOW(), 'Santo Domingo', 'Republica Dominicana')
```

2) ORDENES

```
186 # === ORDENES === #
187
188
189 • INSERT INTO ordenes (id_cliente, fecha_orden) VALUES
190 (1, '2025-01-15 10:20:00'),
191 (2, '2025-02-03 14:45:00'),
192 (3, '2025-03-10 09:30:00'),
193 (4, '2025-04-22 16:10:00'),
194 (5, '2025-05-05 11:50:00'),
195 (6, '2025-06-18 13:25:00'),
196 (7, '2025-07-07 17:40:00'),
197 (8, '2025-08-12 12:15:00'),
198 (9, '2025-09-25 15:30:00'),
199 (10, '2025-10-03 10:05:00'),
200 (11, '2025-11-11 14:20:00'),
201 (12, '2025-12-01 09:55:00'),
202 (13, '2025-01-28 16:45:00'),
203 (14, '2025-02-15 11:10:00'),
204 (15, '2025-03-30 13:50:00'),
205 (16, '2025-04-18 17:25:00'),
206 (17, '2025-05-27 10:40:00'),
```

5) DETALLES ORDEN

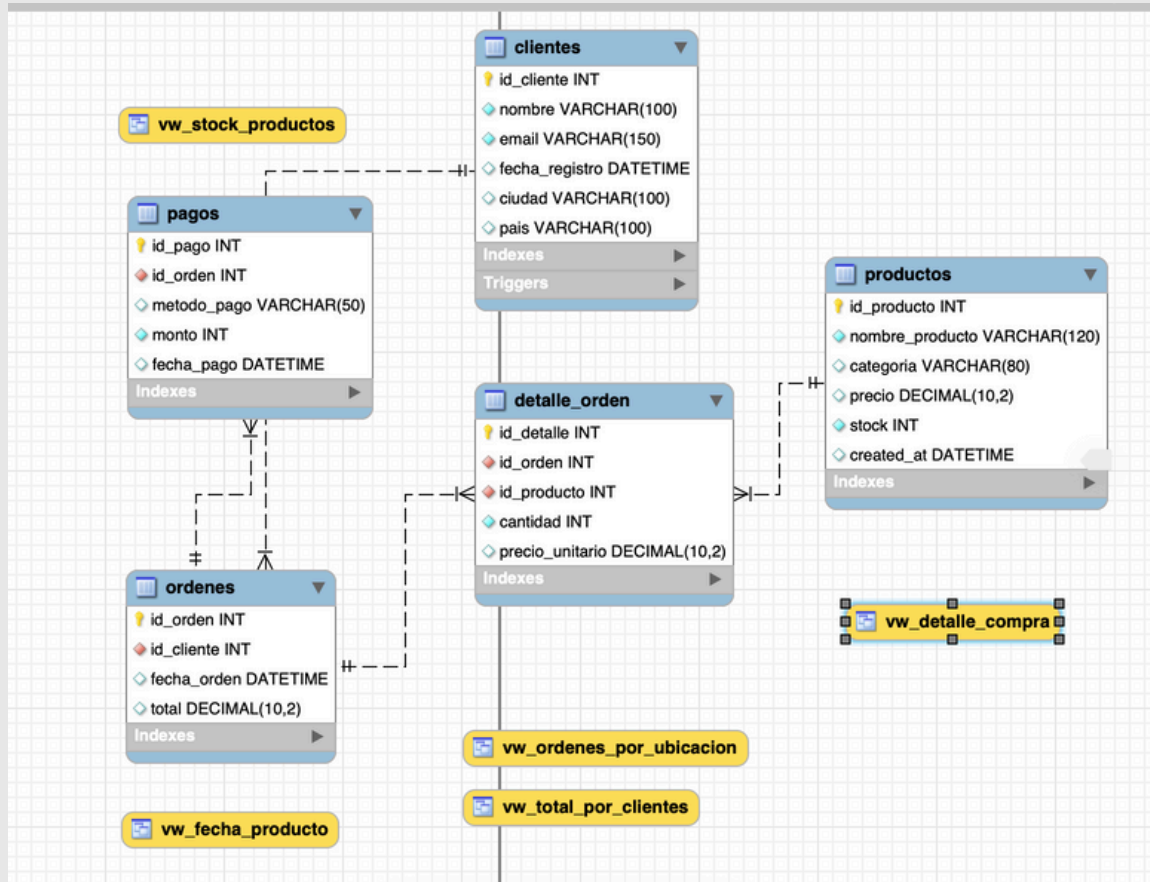
```
215 • INSERT INTO detalle_orden (id_orden, id_producto, cantidad, precio_unitario)
216 VALUES
217 (1, 1, 2, 28.00),
218 (1, 5, 1, 12.00),
219 (2, 2, 1, 18.00),
220 (2, 10, 2, 24.00),
221 (2, 15, 1, 13.00),
222 (3, 3, 1, 22.00),
223 (3, 7, 2, 25.00),
224 (4, 4, 1, 30.00),
225 (4, 12, 1, 14.00),
226 (4, 20, 2, 23.50),
227 (5, 8, 1, 26.00),
228 (5, 10, 2, 19.50),
229 (6, 6, 1, 31.00),
230 (6, 11, 2, 13.50),
231 (7, 9, 1, 21.00),
232 (7, 13, 1, 12.50),
233 (7, 17, 1, 24.50),
234 (8, 14, 2, 28.00),
```

3) PRODUCTOS

```
128 # == PRODUCTOS == #
129
130 INSERT INTO productos (nombre_producto, categoria, precio, stock, created_at) VALUES
131 ('Vinilo "Midnight Echoes" - The Lunar Tides', 'Vinilo', 28.00, 40, '2025-01-05 10:00:00'),
132 ('CD "Midnight Echoes" - The Lunar Tides', 'CD', 18.00, 50, '2025-01-10 11:00:00'),
133 ('Camiseta "Midnight Echoes" - The Lunar Tides', 'Merchandising', 22.00, 80, '2025-01-15 12:00:00'),
134 ('Pin "The Lunar Tides Logo"', 'Merchandising', 6.00, 150, '2025-01-20 13:00:00'),
135 ('Poster "Midnight Echoes Album Art"', 'Poster', 12.00, 60, '2025-01-25 14:00:00'),
136
137 ('Vinilo "Neon Shadows" - Desert Wolves', 'Vinilo', 30.00, 35, '2025-02-05 10:00:00'),
138 ('CD "Neon Shadows" - Desert Wolves', 'CD', 20.00, 45, '2025-02-10 11:00:00'),
139 ('Camiseta "Neon Shadows" - Desert Wolves', 'Merchandising', 24.00, 70, '2025-02-15 12:00:00'),
140 ('Pin "Desert Wolves Logo"', 'Merchandising', 6.50, 140, '2025-02-20 13:00:00'),
141 ('Poster "Neon Shadows Album Art"', 'Poster', 13.00, 55, '2025-02-25 14:00:00'),
142
143 ('Vinilo "Crimson Horizon" - The Velvet Roads', 'Vinilo', 32.00, 30, '2025-03-05 10:00:00'),
144 ('CD "Crimson Horizon" - The Velvet Roads', 'CD', 21.00, 40, '2025-03-10 11:00:00'),
145 ('Camiseta "Crimson Horizon" - The Velvet Roads', 'Merchandising', 25.00, 60, '2025-03-15 12:00:00'),
146 ('Pin "Velvet Roads Logo"', 'Merchandising', 7.00, 120, '2025-03-20 13:00:00'),
147 ('Poster "Crimson Horizon Album Art"', 'Poster', 14.00, 50, '2025-03-25 14:00:00'),
148
149 ('Vinilo "Echoes in Silence" - Midnight Signals', 'Vinilo', 29.00, 38, '2025-04-05 10:00:00'),
```

Comisión 81860
Julian Daud

MODELO ER ACTUALIZADO



Se ajustaron las tablas originales agregando columnas de ubicación y total, y se corrigieron tipos de datos para reflejar mejor los precios y montos, manteniendo las relaciones entre tablas y optimizando la integridad de la base de datos.

VIEWS

Se presentan los resultados de las **vistas** creadas, mostrando que **retornan correctamente la información esperada**.

VIEWS 1 Y 2

```
787 * CREATE VIEW vw_detalle_compra AS
SELECT
  p.categoria,
  pa.metodo_pago,
  d.precio_unitario
FROM detalle_orden AS d
INNER JOIN productos AS p ON d.id_producto = p.id_producto
INNER JOIN ordenes AS o ON d.id_orden = o.id_orden
INNER JOIN pagos AS pa ON o.id_orden = pa.id_orden;

#2=#
CREATE VIEW vw_total_por_clientes AS
SELECT
  c.nombre,
  p.categoria,
  o.fecha_orden,
  pa.monto
FROM clientes AS c
INNER JOIN ordenes AS o ON c.id_cliente = o.id_cliente
INNER JOIN detalle_orden AS do ON o.id_orden = do.id_orden
INNER JOIN productos AS p ON do.id_producto = p.id_producto
INNER JOIN pagos AS pa ON o.id_orden = pa.id_orden;
```

1. **vw_detalle_compra:** Muestra la categoría de los productos, el método de pago y el precio unitario de cada detalle de orden.
Tablas: **detalle_orden, productos, ordenes, pagos**
2. **vw_total_por_clientes:** Muestra el nombre del cliente, categoría de producto, fecha de orden y monto pagado por cada compra. Tablas: **clientes, ordenes, detalle_orden, productos, pagos**

VIEWS 3 Y 4

```
#3=#
CREATE VIEW vw_ordenes_por_ubicacion AS
SELECT
  c.ciudad,
  c.pais,
  COUNT(o.id_orden) AS cantidad_ordenes
FROM clientes AS c
LEFT JOIN ordenes AS o ON c.id_cliente = o.id_cliente
GROUP BY c.ciudad, c.pais;

#4=#
CREATE VIEW vw_fecha_producto AS
SELECT
  o.id_orden,
  o.fecha_orden,
  p.categoria,
  p.created_at
FROM ordenes o
INNER JOIN detalle_orden d ON o.id_orden = d.id_orden
INNER JOIN productos p ON d.id_producto = p.id_producto;
```

1. **vw_ordenes_por_ubicacion:** Agrupa las órdenes por ciudad y país, mostrando la cantidad de órdenes por ubicación. Tablas: **clientes, ordenes**
2. **vw_fecha_producto.** Muestra la fecha de cada orden junto con la categoría del producto y la fecha de creación del producto.
Tablas: **ordenes, detalle_orden, productos**

VIEWS 5

```
325  #=5=#
326
327 • CREATE VIEW vw_stock_productos AS
328 SELECT p.nombre_producto,
329        p.stock,
330        d.precio_unitario
331 FROM productos p
332 JOIN detalle_orden d ON p.id_producto = d.id_producto;
333
```

1. **vw_stock_productos.** Muestra el stock disponible de cada producto junto con el precio unitario registrado en las órdenes. Tablas: **productos**, **detalle_orden**

EJECUCIONES

```
326
327 • CREATE VIEW vw_stock_productos AS
328 SELECT p.nombre_producto,
329        p.stock,
330        d.precio_unitario
331 FROM productos p
332 JOIN detalle_orden d ON p.id_producto = d.id_producto;
333
334
335 # EJECUCIONES #
336
337 • SELECT * FROM vw_detalle_compra
338 WHERE metodo_pago IN ('Venmo');
339
340 • SELECT * FROM vw_total_por_clientes
341 WHERE categoria IN ('Vinilo');
342
343 • SELECT * FROM vw_ordenes_por_ubicacion
344 WHERE pais IN ('Argentina');
345
```

ciudad	pais	cantidad_orden...
Montevideo	Uruguay	3
Brooklyn	USA	2
Buenos Aires	Argentina	2
Madrid	España	1
Cali	Colombia	1
Santiago	Chile	1
Lima	Peru	1
Bogotá	Colombia	1
Quito	Ecuador	1
Asunción	Paraguay	1
La Paz	Bolivia	1

id_orden	fecha_orden	categoria	created_at
1	2025-01-15 10:20:00	Vinilo	2025-12-18 20:40:14
1	2025-01-15 10:20:00	Poster	2025-12-18 20:40:14
2	2025-02-03 14:45:00	CD	2025-12-18 20:40:14
2	2025-02-03 14:45:00	Poster	2025-12-18 20:40:14
3	2025-03-10 09:30:00	Merchandise	2025-12-18 20:40:14
3	2025-03-10 09:30:00	CD	2025-12-18 20:40:14
4	2025-04-22 16:10:00	Merchandise	2025-12-18 20:40:14
4	2025-04-22 16:10:00	CD	2025-12-18 20:40:14
5	2025-05-05 11:50:00	Poster	2025-12-18 20:40:14
5	2025-05-05 11:50:00	Merchandise	2025-12-18 20:40:14

Se muestran consultas de prueba sobre las vistas, funciones y procedimientos almacenados, verificando que devuelven correctamente los datos esperados según la lógica del proyecto.

FUNCIONES

MONTO TOTAL DE ORDENES

```
USE ecommerce_daud;
DELIMITER //

CREATE FUNCTION monto_total_ordenes (param_idOrden INT)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    DECLARE total DECIMAL(10,2);

    SELECT IFNULL(SUM(precio_unitario * cantidad), 0)
    INTO total
    FROM detalle_orden
    WHERE id_orden = param_idOrden;

    RETURN total;
END//

DELIMITER ;

SELECT monto_total_ordenes (2,3,8);
```

The screenshot shows a SQL query result in a web interface. The query is `select ecommerce_daud.monto_total_ordenes(2);`. The result grid shows a single row with the value `79.00`.

ecommerce_daud.monto_total_ordene...
79.00

Calcula el **monto total de una orden** multiplicando precio unitario por cantidad de cada producto en detalle_orden.

Tablas: **detalle_orden**

Objetivo: **Permite obtener rápidamente el total de una orden específica, evitando sumar manualmente los productos.**

UBICACION DE CLIENTES

```
DELIMITER //

CREATE FUNCTION ubicacion_clientes(param_idCliente INT)
RETURNS VARCHAR(250)
DETERMINISTIC
BEGIN
    DECLARE info VARCHAR(250);
    SELECT CONCAT(nombre, ', ', ciudad, ', ', pais) INTO info
    FROM clientes
    WHERE id_cliente = param_idCliente;

    RETURN info;
END//

DELIMITER ;

SELECT ubicacion_clientes (2);

# ---- SP ----
# = 1 = #
```

The screenshot shows a SQL query result in a web interface. The query is `select ecommerce_daud.ubicacion_clientes(10);`. The result grid shows a single row with the value `Ana Fernandez, Bogotá, Colombia`.

ecommerce_daud.ubicacion_clientes...
Ana Fernandez, Bogotá, Colombia

Devuelve el **nombre completo del cliente junto con su ciudad y país** en un solo string. Tablas: **clientes**

Objetivo: **Facilita mostrar información completa del cliente en reportes o vistas sin hacer múltiples joins o concatenaciones.**

STORED PROCEDURES

LISTAR ORDENES DE CLIENTES

```
CREATE PROCEDURE sp_listar_ordenes_cliente (IN p_id_cliente INT)
BEGIN
  SELECT
    o.id_orden,
    o.fecha_orden,
    monto_total_ordenes(o.id_orden) AS total
  FROM ordenes AS o
  WHERE o.id_cliente = p_id_cliente;
END//

DELIMITER //

USE ecommerce_daud;

CALL sp_listar_ordenes_cliente(2);
```

```
1 • call ecommerce_daud.sp_listar_ordenes_cliente(10);
2
```

id_orden	fecha_orden	total
10	2025-10-03 10:05:00	46.50

Muestra todas las órdenes de un cliente específico junto con la fecha y el total de cada orden.

Tablas: **ordenes**, **detalle_orden** (a través de la función **monto_total_ordenes**)

Objetivo: **Permite consultar rápidamente el historial de compras de un cliente sin necesidad de joins complejos.**

ORDENAR CLIENTES

```
DELIMITER //

CREATE PROCEDURE sp_ordenar_clientes(IN p_tabla VARCHAR(225), IN p_columna VARCHAR(225), IN p_orden VARCHAR(225))
BEGIN
  SET @sql_text = CONCAT('SELECT * FROM ', p_tabla, ' ORDER BY ', p_columna, ' ', p_orden, ';');
  PREPARE stmt FROM @sql_text;
  EXECUTE stmt;
  DEALLOCATE PREPARE stmt;
END//

DELIMITER ;

CALL sp_ordenar_clientes('clientes', 'fecha_registro', 'ASC');

=== TRIGGERS ===

DELIMITER //
```

```
1 • call ecommerce_daud.sp_ordenar_clientes('clientes', 'fecha_registro', 'ASC');
2
```

id_cliente	nombre	email	fecha_registro	ciudad	pais
1	Nuevo Cliente	nuevo@email.com	2025-12-18 20:32:16	Montevideo	Uruguay
2	Raul Salvatierra	raul.salvatierra.35@gmail.com	2025-12-18 20:36:16	Brooklyn	USA
3	Maria Perez	maria@hotmail.com	2025-12-18 20:36:16	Buenos Aires	Argentina
4	Carlos Lopez	carlos@gmail.com	2025-12-18 20:36:16	Madrid	España
5	Gian Tenorio	gian.tenorio6@gmail.com	2025-12-18 20:36:16	Call	Colombia
6	Brian Scheller	bischeller@gmail.com	2025-12-18 20:36:16	Brooklyn	USA
7	Juan Perez	juan.perez@gmail.com	2025-12-18 20:36:16	Buenos Aires	Argentina
8	Maria Gonzalez	maria.gonzalez@yahoo.com	2025-12-18 20:36:16	Santiago	Chile
9	Carlos Ramirez	carlos.ramirez@hotmail.com	2025-12-18 20:36:16	Lima	Peru
10	Aria Fernandez	aria.fernandez@gmail.com	2025-12-18 20:36:16	Bogotá	Colombia
11	Luis Martinez	luis.martinez@yahoo.com	2025-12-18 20:36:16	Quito	Ecuador

Ordena dinámicamente cualquier tabla por la columna y el orden especificado.

Tablas: **puede aplicarse a cualquier tabla que se pase como parámetro.**

Objetivo: **Permite listar datos de forma ordenada sin crear múltiples queries fijas, facilitando consultas flexibles y reutilizables.**

TRIGGERS

ACTUALIZAR STOCK

```
439
440 DELIMITER //
441
442 • CREATE TRIGGER trg_actualizar_stock
443   AFTER INSERT ON detalle_orden
444   FOR EACH ROW
445   BEGIN
446     UPDATE productos
447     SET stock = stock - NEW.cantidad
448     WHERE id_producto = NEW.id_producto;
449   END//
450
451 DELIMITER ;
452
453 • SELECT stock FROM productos WHERE id_producto = 15;
454
455
456 DELIMITER //
```

Actualiza automáticamente el **stock de un producto** cada vez que se inserta un nuevo detalle de orden.

Tablas: **detalle_orden, productos**

Objetivo: **Mantener el inventario actualizado sin necesidad de cálculos manuales tras cada venta.**

AGREGAR CLIENTES

```
455
456 DELIMITER //
457
458 • CREATE TRIGGER tr_agregar_cliente
459   AFTER INSERT ON clientes
460   FOR EACH ROW
461   BEGIN
462     INSERT INTO nuevo_cliente(id_cliente, nombre, ciudad, pais)
463     VALUES (NEW.id_cliente, NEW.nombre, NEW.ciudad, NEW.pais);
464   END//
465
466 DELIMITER ;
467
468
469 • INSERT INTO clientes (nombre, email, fecha_registro)
470   VALUES ('Sebastian Cendoya', 'sebastian@example.com', NOW());
471
472
473
```

Inserta automáticamente la información de un **nuevo cliente** en la tabla nuevo_cliente tras cada registro en clientes.

Tablas: **clientes, nuevo_cliente**

Objetivo: **Mantener un registro adicional o histórico de clientes sin tener que hacer inserciones manuales.**